

Reviewer Pack — Geometric Quantization Invariant Bounds Resolution Proof Wid...

Entry fd6762127318 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 03:38:32 UTC

Geometric Quantization Invariant Bounds Resolution Proof Width

Entry ID: fd6762127318 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 03:38:32 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ_G associated with a d -regular graph G , the resolution proof width $w(\varphi_G)$ is bounded by the norm of the corresponding quantum state ρ in the geometric quantization of G , i.e., $w(\varphi_G) \leq \Theta(\text{Norm}(\rho))$.

Rationale (proposer's reasoning):

Geometric quantization assigns a quantum state to a manifold, which encodes information about its geometric properties. Since resolution proof complexity is related to the structure and complexity of the underlying graph, it is plausible that the geometric quantization of this graph could provide insights into the resolution proof width. Furthermore, the norm of the quantum state is computable in polynomial time for compact manifolds, making it a suitable invariant to test.

Taxonomy category: Geometric_Quantization_x_Resolution_Proof_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d0c1b85453b43323

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the seeds show a correlation coefficient $r(\varphi_G, \text{Norm}(\rho)) \geq 0.7$ between resolution proof width $w(\varphi_G)$ and norm of quantum state ρ , where r represents Pearson's correlation coefficient.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -geometric quantization AND resolution proof complexity-Tseitin formula AND d-regular graph AND geometric quantization norm-resolution proof width bounded by quantum state norm in geometric quantization

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        factor = Fraction(-A[i][i], A[i][i])
        for j in range(n):
            if i != j:
                factor = Fraction(-A[j][i], A[i][i])
                for k in range(n+1):
                    A[j][k] += factor * A[i][k]
    return A

def resolution_width(clauses):
    n = len(clauses)
    A = [[0] * (n + 1) for _ in range(n)]
    for i, clause in enumerate(clauses):
        for literal in clause:
            if literal > 0:
                A[i][literal - 1] = 1
            else:
                A[i][-1] += 1
    gaussian_elimination(A)
```

```

width = max(sum(row) for row in A)
return width

def tseitin_formula(graph, d):
    n = len(graph)
    clauses = []
    literals = {}
    for i in range(n):
        literals[i] = random.randint(1, 2*n)
        neg_i = -literals[i]
        clauses.append([literals[i]])
        for j in graph[i]:
            if j != i:
                clauses.append([neg_i, literals[j]])
                clauses.append([neg_i, -literals[j]])
    return clauses

def geometric_quantization(graph):
    n = len(graph)
    rho = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in graph[i]:
            if i != j:
                rho[i][j] = Fraction(1, 2*n)
    return rho

def norm(rho):
    n = len(rho)
    sum_squares = 0
    for i in range(n):
        for j in range(n):
            sum_squares += rho[i][j] ** 2
    return math.sqrt(sum_squares)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            continue

        graph = [[] for _ in range(n)]
        for i in range(n):
            neighbors = random.sample(range(n), d)
            while len(neighbors) < d:
                neighbors.append(random.randint(0, n-1))
            graph[i] = list(set(neighbors))

        clauses = tseitin_formula(graph, d)
        width = resolution_width(clauses)

```

```

rho = geometric_quantization(graph)
norm_rho = norm(rho)

instances_tested += 1
metric_values.append((width, norm_rho))

if instances_tested < 30:
    return {
        "metric_name": "resolution_width",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

correlation_coefficient = 0
for width, norm_rho in metric_values:
    if width == 0 or norm_rho == 0:
        continue
    correlation_coefficient += (width / norm_rho)
correlation_coefficient /= instances_tested

if correlation_coefficient < 0.7:
    conjecture_holds = False
    counterexample = f"correlation_coefficient={correlation_coefficient}"

return {
    "metric_name": "resolution_width",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

```
print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_too_low\" first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cec70ba9.py", line 142, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cec70ba9.py", line 92, in run_trial
    neighbors = random.sample(range(n), d)
                                ^
```

NameError: name 'd' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'd', which prevented the production of data necessary to evaluate the conjecture. | next: Check and define the variable 'd' in the test code, then rerun the test to gather the required data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16789
2	preregistration	ollama_remote	glm4:latest	0	0	18847
3	novelty	ollama_remote	glm4:latest	0	0	17226
4	novelty	ollama_remote	glm4:latest	0	0	16428
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23942
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16968
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27259
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22983
9	judge	ollama_remote	glm4:latest	0	0	9737

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 170179 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/fd6762127318.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fd6762127318.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fd6762127318.tar.gz` (if generated)